

Vizsga sablonok (behelyettesíthető részekkel)

A **__NAGYBETŰ__** részeket kell kicserélni a feladat szerinti nevekre. Mindig a feladatlap szövegében szereplő neveket használd, betűre pontosan.

Jelölés	Mit írj be
__APP__	a Django app neve (pl. camera)
__MODELL__	az új modell neve (pl. Answer)
__RELACIO__	a kapcsolt modell neve (pl. CameraReview)
__VEGPONT__	az API végpont (pl. answers)
__OSZTALY__	a C# osztály neve (pl. Termek)
__FAJL__	a csv / json fájl neve

1. BACKEND (Django)

Előkészítés (a feladatlap szerint, a backend mappában)

```
python -m venv .
Scripts\activate
pip install -r requirements.txt
python manage.py runserver
```

Admin felhasználó

```
python manage.py createsuperuser
```

(felhasználónév és jelszó a feladat szerint, a jelszót kétszer kéri; ha "túl egyszerű", y-nal elfogadható)

models.py

```
class __MODELL__(models.Model):
    kapcsolat = models.ForeignKey(__RELACIO__, on_delete=models.CASCADE)
    nev = models.CharField(max_length=100)
    szoveg = models.CharField(max_length=200)
    szam = models.IntegerField()
    datum = models.DateTimeField()

    def __str__(self):
        return self.nev # ezt kéri a "nickname jelenjen meg" feladat
```

Mezőtípusok: szöveg **CharField(max_length=N)**, egész **IntegerField()**, dátum+idő **DateTimeField()**, idegen kulcs **ForeignKey(Modell, on_delete=models.CASCADE)**. A mezőneveket (**review**, **nickname**, **answer_text**, ...) a feladat szerint add meg.

Migráció

```
python manage.py makemigrations
python manage.py migrate
```

admin.py

```
from .models import __MODELL__
admin.site.register(__MODELL__)
```

Rekord felvétele

Admin felület: <http://127.0.0.1:8000/admin> → a modell → Add.

serializers.py

```
class __MODELL__Serializer(serializers.ModelSerializer):
    class Meta:
        model = __MODELL__
        fields = "__all__"
        depth = 1
```

views.py

```
@api_view(["GET"])
def get__MODELL__s(request):
    osszes = __MODELL__.objects.all()
    serialized = __MODELL__Serializer(osszes, many=True)
    return Response(serialized.data)
```

urls.py (az app-ban)

```
path('__VEGPONT__', views.get__MODELL__s, name="get__MODELL__s"),
```

Ellenőrzés: http://127.0.0.1:8000/api/__VEGPONT__/ (Az `api/` előtagot a projekt `urls.py`-ja adja: `path('api/', include('__APP__.urls'))`.)

2. CSS (exam.css)

```
/* képek lekerekítése */
img {
    border-radius: 5px;
}

/* szöveg szürke, belső span fekete */
.__OSZTALY1__ {
    color: gray;
}
.__OSZTALY1__ span {
    color: black;
}

/* gomb hover (az osztályt Inspect-tel nézd meg) */
.__GOMBOSZTALY__:hover {
    background-color: var(--VALTOZO);
}

/* flexbox, elemek közé teszi a szabad helyet */
.__OSZTALY2__ {
    display: flex;
    justify-content: space-between;
}

/* grid, 2 egyenlő oszlop, 20px távolság */
.__OSZTALY3__ {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 20px;
}

/* reszponzív: a fájl VÉGÉRE */
@media (max-width: 800px) {
    .__OSZTALY4__ {
        grid-template-columns: 1fr;
    }
}
```

Gyakori változatok: `justify-content: center / space-around / space-evenly`, `flex-direction: column`, `align-items: center`, `grid-template-columns: repeat(3, 1fr)`.

3. REACT

Új komponens (components/__KOMPONENS__.jsx)

```
import { useState } from 'react'

function __KOMPONENS__(props) {
    const [szam, setSzam] = useState(props.szam)

    return (
        <div className="__OSZTALY__">
            <strong>{props.nev}</strong>
            <p>{props.szoveg}</p>
            <button onClick={() => setSzam(szam + 1)}>Gomb ({szam})</button>
        </div>
    )
}

export default __KOMPONENS__
```

Dátumnál: `{(new Date(props.datum)).toLocaleDateString()}`

Szülő komponens (lista kirajzolása + lekérés)

```
import __KOMPONENS__ from './__KOMPONENS__.jsx'
import { useState, useEffect } from 'react'
```

```

const __SZULO__ = () => {
  const [__LISTA__, set__LISTA__] = useState([])

  useEffect(() => {
    fetch('/__FAJL__.json')
      .then(response => response.json())
      .then(data => set__LISTA__(data))
  }, [])

  return (
    <div className='__OSZTALY__'>
      {__LISTA__.map(e => (
        <__KOMPONENS__
          key={e.id}
          nev={e.nev}
          szoveg={e.szoveg}
          szam={e.szam}
        />
      ))}
    </div>
  )
}

export default __SZULO__

```

Backendről: `fetch('http://127.0.0.1:8000/api/__VEGPONT__')`.

Ellenőrizd: a `[]` a `useEffect` végén, `key` a `.map()`-ben, a `useState` és `useEffect` importja, a state neve a feladat szerint, a mezőnevek egyeznek a json-nal.

4. KONZOLOS C#

Osztály (__OSZTALY__.cs)

```

internal class __OSZTALY__
{
  public string Nev { get; set; }
  public int Szam { get; set; }
  public string Foglalat { get; set; }

  public __OSZTALY__(string sor)
  {
    string[] m = sor.Split(';');
    Nev = m[0];
    Szam = int.Parse(m[1]);
    Foglalat = m[2];
  }

  // logikai metódus (pl. "Kiemelt")
  public bool Kiemelt()
  {
    return Foglalat == "E27" && Szam == 40;
  }
}

```

A tulajdonságok sorrendje a csv fejlécét követi. Ha egy mező nem csak szám, `string` kell.

Program.cs

```

using System.Globalization;
using System.Text;

// beolvasás (az első sor a fejléc)
List<__OSZTALY__> lista = new List<__OSZTALY__>();
foreach (string sor in File.ReadAllLines("__FAJL__.csv", Encoding.UTF8).Skip(1))
{
  lista.Add(new __OSZTALY__(sor));
}

// darabszám
Console.WriteLine("4. Feladat: A lámpák száma: {0} db", lista.Count);

// szűrés két feltétellel
Console.WriteLine("5. Feladat: ...");
foreach (__OSZTALY__ x in lista.Where(x => x.Foglalat == "GU5.3" && x.Szam == 6000))
{
  Console.WriteLine(x.Nev);
}

```

```

}

// átlag billentyűzetről beolvasott érték alapján
Console.WriteLine("7. feladat: Kérem a foglalat nevét: ");
string be = Console.ReadLine();
var talalat = lista.Where(x => x.Foglalat.ToLower() == be.ToLower()).ToList();
if (talalat.Count > 0)
{
    double atlag = talalat.Average(x => x.Szam);
    Console.WriteLine("A lámpák átlagos teljesítménye: " + atlag.ToString("0.0", new CultureInfo("hu-HU")));
}
else
{
    Console.WriteLine("Nem található ilyen foglalatú lámpa.");
}

```

A csv fájl tulajdonsága: Copy to Output Directory → Copy if newer. A kírásokat a feladat mintája szerint másold. Hasznos LINQ: `.Count(x => ...)`, `.Max(x => ...)`, `.Min(x => ...)`, `.Sum(x => ...)`, `.OrderBy(x => ...)`, `.Any(x => ...)`, `.FirstOrDefault(x => ...)`.

5. GRAFIKUS C# (WPF)

NuGet

Jobb gomb a projekten → Manage NuGet Packages → `MySQL.Data` (vagy ami a `forras.txt`-ben szerepel).

MainWindow.xaml (a Grid belseje; az ablak Title-je a minta szerint)

```

<Grid>
    <DataGrid x:Name="dgLista"
        AutoGenerateColumns="False"
        IsReadOnly="True"
        SelectionMode="Single"
        HorizontalAlignment="Left"
        Width="500"
        Margin="10">
        <DataGrid.Columns>
            <DataGridTextColumn Header="Név" Binding="{Binding Nev}" Width="*" />
            <DataGridTextColumn Header="Foglalat" Binding="{Binding Foglalat}" Width="Auto" />
        </DataGrid.Columns>
    </DataGrid>

    <StackPanel HorizontalAlignment="Left" Margin="520,10,0,0" Width="250">
        <Button x:Name="btnElso" Content="4. feladat" Click="btnElso_Click" Margin="0,0,0,5" />
        <Label x:Name="lblElso" Content="Kategória: " Margin="0,0,0,20" />

        <Button x:Name="btnMasodik" Content="5. feladat" Click="btnMasodik_Click" Margin="0,0,0,5" />
        <Label x:Name="lblMasodik" Content="" />
    </StackPanel>
</Grid>

```

A `Binding` neveknek egyezniük kell a `Lampa.cs` tulajdonságaival (betűre pontosan).

MainWindow.xaml.cs

```

using MySql.Data.MySqlClient;

public partial class MainWindow : Window
{
    List<Lampa> lista = new List<Lampa>();
    // a kapcsolati szöveget a forras.txt-ből másold ki
    MySqlConnection connection = new MySqlConnection("server=localhost;database=__ADATBAZIS__;uid=root;password=''");

    public MainWindow()
    {
        InitializeComponent();

        connection.Open();
        MySqlCommand cmd = new MySqlCommand("SELECT * FROM __TABLA__", connection);
        using (MySqlDataReader reader = cmd.ExecuteReader())
        {
            while (reader.Read())
            {
                lista.Add(new Lampa(reader)); // a Lampa.cs konstruktora szerint
            }
        }
        connection.Close();
    }
}

```

```

        dgLista.ItemsSource = lista;
        dgLista.SelectedIndex = 0;
    }

    // a kijelölt elem adata
    private void btnElso_Click(object sender, RoutedEventArgs e)
    {
        Lampa kivlasztott = dgLista.SelectedItem as Lampa;
        lblElso.Content = "Kategória: " + kivlasztott.Kategoria;
    }

    // darabszám feltétellel
    private void btnMasodik_Click(object sender, RoutedEventArgs e)
    {
        int db = lista.Count(x => x.Elettartam >= 5000);
        lblMasodik.Content = "Legalább 5000 élettartamú: " + db;
    }
}

```

Ha a **Lampa** osztályban nincs kategória, a **SELECT**-ben kell egy **INNER JOIN**: `SELECT t.termekNev, t.foglalat, t.elettartam, k.katNev FROM termek t INNER JOIN kategoriak k ON t.kategoriaId = k.katId`

Adatbázis (phpMyAdmin)

```

CREATE DATABASE __ADATBAZIS__
CHARACTER SET utf8mb4
COLLATE utf8mb4_hungarian_ci;

```

Utána: kijelölsz az adatbázist → Importálás → a **.sql** fájl → Végrehajtás.

Ellenőrző lista a vizsga végén

- A nevek (projekt, osztályok, state-ek, végpont, felhasználónév) betűre pontosan a feladat szerint vannak.
- A kiírások a mintát követik.
- Backend: migrate lefutott, az API-ban látszik a rekord.
- React: nincs piros hiba a konzolban, a lista megjelenik.
- CSS: a `@media` a fájl végén van.
- C#: a csv a kimeneti mappába másolódik, a fejléct kihagyod.